

AIM

MD.040 BUILD STANDARDS

<Company Long Name>

<Subject>

Author: <Author>
Creation Date: April 24, 1999
Last Updated: August 15, 2025
Document Ref: <Document Reference Number>
Version: DRAFT 1A

Approvals:

<Approver 1>

<Approver 2>

ORACLE®

Copy Number _____

Document Control

Change Record

Date	Author	Version	Change Reference
24-Apr-99	<Author>	Draft 1a	No Previous Document

Reviewers

Name	Position

Distribution

Copy No.	Name	Location
1	Library Master	Project Library
2		Project Manager
3		
4		

Note To Holders:

If you receive an electronic copy of this document and print it out, please write your name on the equivalent of the cover page, for document control purposes.

If you receive a hard copy of this document, please write your name on the front cover, for document control purposes.

Contents

Document Control	ii
Introduction	1
Purpose	1
Background.....	1
Scope and Application	1
Related Documents.....	1
The Development Environment	2
Directory Structure	2
Registering Your Application	3
Custom Tables.....	3
Your Development Account.....	3
Century Date Compliance	4
Building Your Programs.....	5
Registering Your Programs	5
Common Standards	6
Forms Coding Standards	7
Exceptions to Oracle Standards	7
Report Coding Standards	8
SQL Standards.....	9
Exceptions to Oracle Standards	9
PL/SQL Standards	10
Exceptions to Oracle Standards	10
Database Trigger Standards	11
Other Coding Standards	12
Comment Standards.....	13
Installation Routine Standards.....	14
Basic Business Needs.....	14
Script Organization	14
Oracle Seed Data Loaders.....	16
Source Code Control	17
Using RCS.....	17

Open and Closed Issues for this Deliverable	18
Open Issues.....	18
Closed Issues	18

Introduction

Purpose

This document describes the standards that <Company Long Name> will follow on the <Project Name> when building customizations to the Oracle Applications.

Background

The information in this document has been defined as the result of discussions between project team representatives, <Company Short Name> technical staff, and <Consulting Services Provider> consultants.

Scope and Application

The standards in this document cover the Build phase and will primarily affect tasks in the Module Design and Build (MD) process of AIM:

Related Documents

1. Customization Strategy for <Project Name>.
2. Design Standards for <Project Name>.
3. *Oracle Applications Coding Standards*
4. *Building Systems using ORACLE Programming Tools*

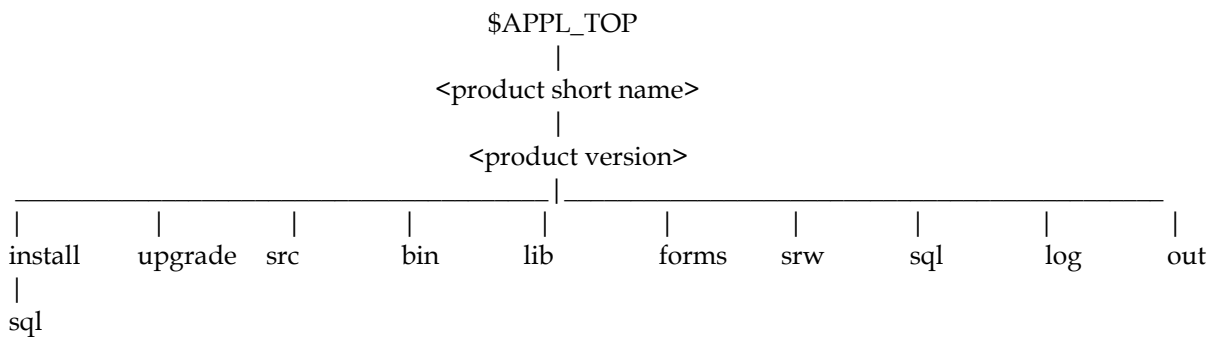
The Development Environment

Each product (GL, Purchasing, Inventory, etc.) in an installation of Oracle Applications has an associated operating system directory. All files required to install, upgrade, or run the product are stored in this directory (or related subdirectories). Each product also has one or more database user accounts which own tables used by the product. Application Object Library calls this collection of objects an *application*.

A set of customizations for one or more products is installed as an application. Thus it has a product directory and an Oracle user account.

Directory Structure

The directory structure for an installed product is as follows:



If you wish to create regression tests, you will need a *regress* directory. Some products may have an *rpt* directory for SQL*Report programs, but Oracle Reports 2.0 is the current standard for all new reports.

The product short name is a two to eight character abbreviation of the product (i.e., GL, PO, INV, etc.). The short name for <Company Short Name> custom applications will be <Application Prefix (UPPERCASE)> followed by the application short name that the customizations are for such as <Application Prefix (UPPERCASE)>GL, and <Application Prefix (UPPERCASE)>INV.

The product version subdirectory level allows multiple versions of a product to be available within a single Applications installation. The naming convention is <major version>.<minor version>.<patch level> (i.e. 4.1.0). The standard is to begin with 1.0.0 for a new product regardless of the general release number or Oracle Applications.



Oracle Applications Installation Manual
Application Object Library Reference Manual

Registering Your Application

You must register your application before you can begin developing custom modules. The following is an example of the steps typically required to register a new application:

1. Create the directory structure
2. Modify \$APPL_TOP/APPLSYS.env to set the environment variable <shortname>_TOP and add \$<shortname>_TOP/bin to the PATH environment variable.

Example:

```
<Application Prefix (UPPERCASE)>GL_TOP=$APPL_TOP/<Application Prefix (lowercase)>gl/1.0.0
export <Application Prefix (UPPERCASE)>GL_TOP
PATH=$PATH:$<Application Prefix (UPPERCASE)>GL_TOP/bin
Export PATH
```

3. Shutdown the concurrent manager.
4. Execute the APPLSYS.env file.
 . APPLSYS.env
5. Restart the concurrent manager.
6. Invoke found and select the Application Developer responsibility
7. Register the application

Custom Tables

New tables required by your custom programs should be created in an Oracle user created exclusively for customizations. For convenience, it should match the application short name of your custom application (i.e., <Application Prefix (UPPERCASE)>GL). Grants and synonyms will allow other Oracle users to access your custom tables.

If you plan to create a new responsibility to access custom forms exclusively, you must register your custom Oracle id with Application Object Library. If your forms will be accessed from responsibilities that also access standard forms, it is easier to simply create grants and synonyms so that the Oracle user associated with the responsibility can access your tables.



Attention: In Applications Release 10.6 and greater, all users access tables from the APPS Oracle id. You must create grants and synonyms so your tables are available in the APPS Oracle id.

Your Development Account

Develop and test custom modules in your home directory. The easiest way to do this is to create the directories: forms, bin, src, srw, sql, log, and out (log and out can be soft links to the actual log and out directories). Reset the product top environment variable for your custom application to point to your home directory and invoke the application with the “found” command. In UNIX, this is easiest with a simple shell script:

```
#!/bin/sh
# testfound - run found to test programs locally
#
<Application Prefix (UPPERCASE)>GL_TOP=$HOME ; export
<Application Prefix (UPPERCASE)>GL_TOP
found
```



Suggestion: If multiple custom directories exist (e.g., one for customizations to Oracle General Ledger and another for Order Entry) create subdirectories in your home directory (e.g., gldev, oedev). Then create the proper directories in each. You will need a different test script for each set of customizations.

If you needed to modify a report written in SQL*Plus, the sequence of UNIX commands would be:

```
cd sql
cp $<Application Prefix
(UPPERCASE)>GL_TOP/sql/myreport.sql .
vi myreport.sql

      (make necessary changes)

testfound

      (test the changes)

mv myreport.sql $<Application Prefix
(UPPERCASE)>GL_TOP/sql
```

If your programs are under source control the process is slightly different:

```
cd sql
co -l myreport.sql
vi myreport.sql

      (make necessary changes)

testfound

      (test the changes)

ci -u myreport.sql
mv myreport.sql $<Application Prefix
(UPPERCASE)>GL_TOP/sql
```



See the Source Code Control section at the end of this document.

Century Date Compliance

In the past, two character date coding was an acceptable convention due to perceived costs associated with the additional disk and memory storage requirements of full four character date encoding. As the year 2000 approached, it became evident that a full four character coding scheme was more appropriate.

In the context of the Application Implementation Method (AIM), the convention Century Date or C/Date support rather than Year2000 or Y2K support is used. Coding for any future Century Date is now considered the modern business and technical convention.

Every applications implementation team needs to consider the impact of the century date on their implementation project. As part of the implementation effort, all

customizations, legacy data conversions, and custom interfaces need to be reviewed for Century Date compliance.

When designing and building application extensions, it is essential that all dates be entered, stored, and processed using the full four digit year for compliance with Century Date standards. In the case of custom interfaces, both the program code and imported legacy or third-party application data must be checked for compliance with Century Date standards.

Building Your Programs

Develop and test your programs from within your local directories. Follow the instructions in the reference manual of each Oracle tool you plan to use to assist in making certain your environment is properly initialized.

Any text editor can be used for C programs, SQL*Plus, and PL/SQL programs, we have standardized on <Standard Text Editor> for editing source code.

Registering Your Programs

Follow the instructions in the *Application Object Library Reference Manual* to register your forms and programs. All programs must be registered in order to be attached to menus or executed in the concurrent manager. Concurrent programs and reports do not need to be reregistered after they are modified. Forms only need to be reregistered if you add, delete, or change the order of fields or zones.



Application Object Library Reference Manual.

Common Standards

Standard File Headers

Comments

Forms Coding Standards

Oracle Forms coding standards are governed by the standards published by Oracle in *Oracle Applications User Interface Standards*.

Exceptions to Oracle Standards

Follow all standards defined in the Oracle standards manuals except for the following exceptions:

- -
 -
-

Report Coding Standards

Structure and Style

Debugging Techniques

Variable Naming and Usage

Performance Improvement Techniques

Exception Handling

Error Messages

Porting Considerations

SQL Standards

The primary standards for SQL standards is described in *Building Systems using ORACLE Programming Tools*.

Exceptions to Oracle Standards

Follow all standards defined in the Oracle standards manuals except for the following exceptions:

- -
 -
-

PL/SQL Standards

The primary standards for SQL standards is described in *Building Systems using ORACLE Programming Tools*.

Exceptions to Oracle Standards

Follow all standards defined in the Oracle standards manuals except for the following exceptions:

- -
 -
-

Database Trigger Standards

Structure and Style

Database triggers must follow the general standards for PL/SQL.

Performance Improvement Techniques

To improve performance, include all trigger logic in a stored procedure and call the stored procedure from your database trigger.

Other Coding Standards

Comment Standards

Installation Routine Standards

Standard Oracle Applications use Autoinstall to create the database objects and seed data that they need to run. Customizations cannot use Autoinstall and must use another method.

Basic Business Needs

The scripts that you create to install your customization should:

- Install all objects with the minimum number of manual steps.
- Be reexecutable without errors or creating duplicate data.
- Be transportable to a different environment or machine.

Script Organization

Each customization will have one master operating script to create all database objects and perform other operations to install the modules of the customization. The script name will use the customization identifier followed by `_inst.sh` (for Unix shell scripts). The customization identifiers are established during Business Requirements Mapping and are documented on the Application Gap List.

The operating system script will call all of the database object creation scripts required of the customization. All database object creation scripts will use the `CREATE OR REPLACE <object name>` syntax so they can be rerun on demand. The master script must also call a grant/synonym script if it requires access to database objects outside of the Oracle ID in which it runs. Scripts may also launch concurrent programs to register program modules or load interface tables as needed.

Seed Data

A single script can insert the required lookup codes for all customizations. Forms will reference this lookup table for List of Values (LOV) and QuickCodes instead of FND_LOOKUPS (except when you need to access a standard lookup code). Each distinct lookup code must have its own insert statement. Each insert statement uses the "Where not exists..." clause to insert data only if the data does not already exist. You should also add comments indicating which customization each set of lookup codes is used for, as in the example below:

```
REM      <Application Prefix (lowercase)>gllkps - Seed lookups for
custom product <Application Prefix (UPPERCASE)>GL
REM
REM      Usage: sqlplus <Application Prefix
(lowercase)>gl_un/<Application Prefix (lowercase)>gl_pw @<Application
Prefix (lowercase)>gllkps.sql
REM
insert into <Application Prefix (lowercase)>gl_lookups
      (LOOKUP_TYPE      ,
LOOKUP_CODE      ,
LAST_UPDATE_DATE  ,
LAST_UPDATED_BY   ,
```

```

        CREATION_DATE      ,
        CREATED_BY         ,
        LAST_UPDATE_LOGIN,
        MEANING            ,
        ENABLED_FLAG       ,
        START_DATE_ACTIVE,
        END_DATE_ACTIVE)
SELECT
    '****',                * Enter lookup_type
    '****',                * Enter lookup_code
    SYSDATE,
    -1,
    SYSDATE,
    -1,
    -1,
    '****',                * Enter meaning here
    'Y',
    NULL,
    NULL
from dual
where not exists (select 'X' from <Application Prefix
(lowercase)>gl_lookups
                  where lookup_type = '****'          * Enter
same lookup_type and lookup_code = '****');          * Enter
same lookup_code

REM copy and modify the above statement for each required
lookup code

commit;
exit;

```

Grants and Synonyms

Custom tables must be accessible from the account associated with the responsibility that custom menu entries are attached to. For example, if you add your custom forms to menus in the Oracle Manufacturing responsibility, your grant script should grant access to and create synonyms in the MFG Oracle user id. The source and destination user ids and passwords are passed as parameters to the script, as in the example below:

```

REM Create grants/synonyms for all custom tables to APPS
account
REM sqlplus &un/&pw @<Application Prefix (lowercase)>glgtmfg.sql
&un_app &pw_app
REM
connect &1/&2
grant all on <Application Prefix (UPPERCASE)>GL_LOOKUPS
to APPS;
grant all on <Application Prefix (UPPERCASE)>GL_TABLE_1
to APPS;
connect APPS/APPS
create or replace synonym <Application Prefix
(UPPERCASE)>GL_LOOKUPS
for &1..<Application Prefix
(UPPERCASE)>GL_LOOKUPS;
create or replace synonym <Application Prefix
(UPPERCASE)>GL_TABLE_1

```

```
                                for &1..<Application Prefix  
(UPPERCASE)>GL_TABLE_1;  
commit;  
exit;
```

Oracle Seed Data Loaders

Oracle Applications include the following to load or migrate seed data and other configuration information into an instance of Oracle Applications.

- Function Security Loader
- Message Dictionary Generator
- User Profile Loader
- User Profile Value Loader
- Concurrent Program API
- Concurrent Manager API
- Descriptive Flexfield API
- Descriptive Flexfield Value Set API



Attention: Consult your Applications documentation for detailed instructions on using these utilities.

Source Code Control

Source code control is the process of managing revisions to program source code and the modification of files by multiple individuals. On Unix systems the two most common utilities are SCCS (Source Code Control System) and RCS (Revision Control System). Oracle Vertical Applications uses ARCS (Application Revision Control System), a modified version of RCS. RCS is the easiest of these to use.

Rather than describe RCS in detail, consult the Unix manual pages for RCS, CO, and CI.

Using RCS

To simplify the maintenance of RCS files, an RCS directory should be created in each directory where source code files are stored. Normally, this will be forms, src, srw, and sql. Also, each developer should create a soft link from subdirectories in his or her home directory to the corresponding RCS files in the custom product directory. For example:

```
/home/jlange
      forms/RCS           linked to ---> $<Application Prefix
(UPPERCASE)>GL_TOP/forms/RCS
      src/RCS            linked to ---> $<Application Prefix
(UPPERCASE)>GL_TOP/src/RCS
```

This allows you to use the co and ci commands without specifying the full pathname of the workfile. For example, to begin working on a form called <Application Prefix (UPPERCASE)>GLNEWFRM, issue the commands:

```
cd forms
co -l <Application Prefix (UPPERCASE)>GLNEWFRM.inp
```

After testing, check the form back in and install it with:

```
ci -u <Application Prefix (UPPERCASE)>GLNEWFRM.inp
mv <Application Prefix (UPPERCASE)>GLNEWFRM.* $<Application
```

Prefix (UPPERCASE)>GL_TOP/forms

The -u option of ci checks the file back in and unlocks it, but then immediately checks it back out without a lock. This allows you to copy the current source and compiled versions of the code into the normal forms directory. Strictly speaking, keeping the source file in the forms directory is redundant since the current revision can always be extracted from the rcs workfile. For some programs such as SQL*Plus scripts, the source code is also the executable and must be in the proper directory for Application Object Library to find it.

Open and Closed Issues for this Deliverable

Open Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date

Closed Issues

ID	Issue	Resolution	Responsibility	Target Date	Impact Date